

Documentazione: Sistema Gestione Ospedale

Gada Mykhaylo, Giordano Francesco Maria
Gruppo 27

2026

Indice

1	Introduzione	2
2	Architettura dei Package	2
3	Il Package model	2
3.1	Gestione delle Utenze e dei Ruoli	2
3.2	Gestione Strutturale	2
3.3	Processi Clinici e Amministrativi	3
4	Il Package controller	3
5	Il Package gui	3
5.1	Finestre Principali e Autenticazione	3
5.2	Schermate Operative	3
6	Il Package dao	4
7	Il Package implementazioneDao	4
8	Il Package database_connection	5
8.1	Gestione delle transazioni	5
9	Il Package util e la Crittografia	6
9.1	Gestione della Sicurezza e PasswordHasher	6

1 Introduzione

Il presente documento descrive l'architettura software del progetto di "Sistema di Gestione Ospedale". L'applicazione è strutturata secondo un'architettura multilivello che separa la logica di dominio, il flusso di controllo e l'interfaccia utente.

2 Architettura dei Package

I package fondamentali analizzati in questo documento sono:

- **model**: Contiene le entità del dominio.
- **controller**: Gestisce il flusso dell'applicazione e la comunicazione tra l'interfaccia utente e il modello.
- **gui**: Contiene le classi dedicate all'Interfaccia Grafica Utente (Graphical User Interface).
- **dao**: Definisce le interfacce astratte per l'accesso ai dati.
- **implementazioneDao**: Contiene le implementazioni concrete delle interfacce DAO che eseguono le query SQL sul database PostgreSQL.
- **database_connection**: Gestisce il livello infrastrutturale a basso livello per l'apertura, la configurazione e il rilascio delle connessioni JDBC.

3 Il Package model

Il package **model** rappresenta il cuore del sistema e implementa le entità reali del dominio clinico/ospedaliero.

3.1 Gestione delle Utenze e dei Ruoli

- **Utente**: Classe astratta (o classe base) che incapsula i dati anagrafici e le credenziali di accesso al sistema.
- **Medico**: Estende **Utente**. Aggiunge nome, cognome e lo stato di disponibilità del sanitario, oltre ai riferimenti ai turni di lavoro assegnati (**TurnoLavorativo**) e allo storico delle prestazioni erogate.
- **Amministratore**: Estende **Utente**. Rappresenta gli utenti con privilegi elevati (personale amministrativo) autorizzati a gestire infrastruttura e personale.
- **Paziente**: Rappresenta il paziente ospedaliero con i propri dati anagrafici (codice fiscale, nome, cognome, recapito).

3.2 Gestione Strutturale

Queste classi modellano la struttura fisica dell'ospedale:

- **Reparto**: Modella una singola unità ospedaliera (es. Cardiologia, Chirurgia), contenente l'elenco delle stanze che la compongono.
- **Stanza**: Un'unità all'interno di un reparto, contenente una lista o una serie di letti.
- **Letto**: Rappresenta la singola postazione in cui un paziente può essere ricoverato. Contiene lo stato di occupazione.

3.3 Processi Clinici e Amministrativi

- **Ricovero**: Gestisce le informazioni relative a un'ammissione, associando un **Paziente** (tramite codice fiscale) a un **Letto** per un periodo definito da data di ricovero e data di dimissione (prevista ed effettiva). Include inoltre diagnosi di entrata e di uscita, eventuale terapia prescritta, una descrizione libera, l'indicazione se si tratti di un ricovero in regime di Day Hospital, e lo stato corrente (in corso o concluso).
- **PrestazioneMedica**: Entità che modella una visita, un intervento o un esame svolto da un Medico per un Paziente.
- **TurnoLavorativo**: Gestisce gli orari e la turnazione del personale sanitario, definendo per ciascun turno il giorno della settimana e gli orari di inizio e fine.

4 Il Package controller

Il package `controller` funge da intermediario tra l'interfaccia grafica e i dati applicativi (inclusi l'interfacciamento con i layer DAO per la persistenza dei dati).

- **Controller**: Il controller di base, che espone le operazioni comuni o generali e fa da snodo per le operazioni applicative di base (es. autenticazione iniziale).
- **AdminController**: Gestisce esclusivamente le operazioni di competenza dell'amministratore (assegnazione di letti, gestione del personale, gestione dell'anagrafica pazienti, registrazione di ricoveri e dimissioni, e visualizzazione dello stato di occupazione dei letti per reparto).
- **MedicoController**: Contiene la logica operativa dedicata ai medici, come la registrazione di ricoveri, la creazione di prestazioni mediche per i pazienti a loro assegnati e le dimissioni.

5 Il Package gui

Il package `gui` racchiude l'interfaccia visiva con la quale l'utente interagisce.

5.1 Finestre Principali e Autenticazione

- **Login**: La schermata di avvio del sistema. Raccoglie le credenziali dell'utente e, tramite il controller, indirizza verso la corretta interfaccia di competenza (Medico o Amministratore).
- **InterfacciaAmministratore**: Dashboard dedicato agli utenti di tipo Admin, dal quale si diramano i menu per le gestioni strutturali.
- **InterfacciaMedico**: Dashboard stabile per i medici, per visionare turni, pazienti e attività mediche assegnate.

5.2 Schermate Operative

- **GestioneAnagraficaPazienti**: Interfaccia per inserire, modificare o consultare le schede anagrafiche dei pazienti.
- **GestioneLetti**: Form per la visualizzazione, filtrabile per reparto o per codice, dello stato di occupazione dei letti.
- **RegistraRicovero**: Schermata utilizzata per effettuare l'accettazione e l'immatricolazione di un ricovero.

- **RegistraDimissione**: Permette al personale sanitario di registrare l'uscita o il rilascio di un paziente e liberare il relativo letto.
- **RegistraPrestazione**: Usata dal medico per refertare e inserire i dettagli di una visita o intervento a sistema.
- **PanelImmagine**: Classe di supporto utility che estende `JPanel` per permettere un facile caricamento e scaling di grafiche nell'interfaccia (es. `logoFMAB.png` trovato nel package `Resources`).

6 Il Package dao

Il package `dao` (Data Access Object) definisce il livello di astrazione per l'accesso ai dati. Utilizzando questo design pattern, il sistema separa la logica del model dalle specifiche operazioni di persistenza, definendo interfacce che stabiliscono le firme dei metodi senza preoccuparsi dell'implementazione del database sottostante.

- **LettoDAO**: Interfaccia che espone i metodi per la ricerca, l'aggiornamento dello stato (es. libero/occupato) e il salvataggio dei letti.
- **PazienteDAO**: Definisce le operazioni per l'inserimento di nuovi pazienti, l'aggiornamento dei loro dati anagrafici e la ricerca all'interno del database.
- **PrestazioneMedicaDAO**: Contiene le signature dei metodi per registrare, aggiornare e recuperare lo storico delle prestazioni mediche erogate ai pazienti.
- **RepartoDAO**: Interfaccia per l'accesso alle informazioni sui reparti ospedalieri, utile per popolare menu a tendina o visualizzare statistiche globali.
- **RicoveroDAO**: Definisce le operazioni necessarie per gestire il ciclo di vita di un ricovero (creazione, aggiornamento per dimissione e recupero storico).
- **StanzaDAO**: Interfaccia per il recupero e la manipolazione dei dati delle stanze all'interno di un dato reparto.
- **TurnoLavorativoDAO**: Definisce i metodi per il salvataggio e il recupero dei turni assegnati al personale medico e amministrativo.
- **UtenteDAO**: Interfaccia fondamentale per la gestione delle credenziali; espone metodi per recuperare gli utenti in fase di login e per gestirne l'anagrafica di base.

7 Il Package implementazioneDao

Questo package contiene le implementazioni concrete delle interfacce definite nel package `dao`. Nello specifico, le classi presenti interagiscono direttamente con un database relazionale PostgreSQL, eseguendo le query SQL necessarie.

- **LettoPostgresDao**: Implementa `LettoDAO`. Mappa gli oggetti `Letto` in tuple relazionali ed esegue query di `UPDATE` per modificare l'occupazione.
- **PazientePostgresDao**: Implementa `PazienteDAO`. Gestisce query di `INSERT` e `SELECT` per interagire con la tabella dei pazienti.
- **PrestazioneMedicaPostgresDao**: Implementa `PrestazioneMedicaDAO`. Traduce i referti e i dati clinici in istruzioni SQL per il salvataggio a database.

- **RepartoPostgresDao**: Implementa **RepartoDAO**. Estrae dal database le gerarchie strutturali (reparto → stanza → letto).
- **RicoveroPostgresDao**: Implementa **RicoveroDAO**. Gestisce le operazioni SQL relative al ciclo di vita del ricovero (creazione, aggiornamento in fase di dimissione, recupero storico per paziente).
- **StanzaPostgresDao**: Implementa **StanzaDAO**. Esegue le query necessarie per enumerare le stanze e filtrarle per reparto.
- **TurnoLavorativoPostgresDao**: Implementa **TurnoLavorativoDAO**. Salva e interroga il database per ottenere gli orari e le presenze del personale.
- **UtentePostgresDao**: Implementa **UtenteDAO**. Responsabile della query di autenticazione e dell'estrazione dei ruoli (**Medico** o **Amministratore**) interrogando le relative tabelle del database PostgreSQL.

8 Il Package `database_connection`

Il package `database_connection` gestisce il livello più basso dell'infrastruttura dati, occupandosi dell'apertura e della chiusura sicura delle connessioni verso il DBMS.

- **ConnessioneDatabase**: Classe (implementata secondo il pattern Singleton) che si occupa di istanziare e mantenere un'unica connessione JDBC (Java Database Connectivity) condivisa verso PostgreSQL, tramite il connection pooler offerto da Supabase. La connessione viene riutilizzata per tutta la durata dell'applicazione e non viene mai chiusa esplicitamente da questa classe: sono le singole implementazioni DAO a essere responsabili della chiusura dei propri Statement e ResultSet tramite blocchi try-with-resources, per evitare memory leak senza invalidare la connessione condivisa. La classe fornisce inoltre un meccanismo per eseguire più operazioni come un'unica transazione atomica (vedi paragrafo 8.1), garantendo la coerenza dei dati quando un'azione utente richiede più scritture correlate sul database.

8.1 Gestione delle transazioni

Alcune operazioni applicative richiedono che più scritture sul database avvengano in modo atomico: ad esempio, la registrazione di un ricovero comporta sia l'aggiornamento dello stato di un letto (da libero a occupato) sia l'inserimento del nuovo ricovero, mentre una dimissione comporta sia la chiusura del ricovero sia la liberazione del letto. Se una delle due operazioni fallisse dopo che l'altra è già stata confermata, il database si troverebbe in uno stato incoerente (un letto bloccato senza ricovero, o viceversa). Per evitare questo scenario, **ConnessioneDatabase** espone un metodo che disattiva temporaneamente l'autocommit della connessione, esegue le operazioni richieste, e conferma (commit) o annulla (rollback) entrambe insieme in base al loro esito complessivo, ripristinando infine il comportamento normale della connessione.

9 Il Package util e la Crittografia

Il package `util` contiene classi di utilità generale che offrono servizi trasversali a tutta l'applicazione, non strettamente legati alla logica di dominio ospedaliero.

9.1 Gestione della Sicurezza e PasswordHasher

- **PasswordHasher**: Questa classe fornisce metodi statici essenziali per garantire la sicurezza del sistema, implementando funzioni di crittografia per la protezione delle credenziali.

In conformità con le best practices di sicurezza informatica, le password degli utenti non vengono mai salvate o confrontate in chiaro nel database. La classe `PasswordHasher` si occupa di due mansioni principali:

- **Hashing**: Durante la creazione di un nuovo account o il cambio password, prende la password in chiaro inserita e la converte in un hash unidirezionale tramite l'algoritmo BCrypt (libreria `org.mindrot.jbcrypt`), che incorpora automaticamente un salt casuale per ogni password.
- **Verifica (Match)**: Durante il processo di login nella schermata `Login`, la password inserita dall'utente viene passata a questa classe, che la espande in un hash e lo confronta con la stringa crittografata recuperata tramite l'`UtentePostgresDao`. Solo in caso di corrispondenza il sistema autorizza l'accesso.